ACUITEY SOLUTIONS • ENTERPRISE ARCHITECTURE

Acuitey Solutions LMS

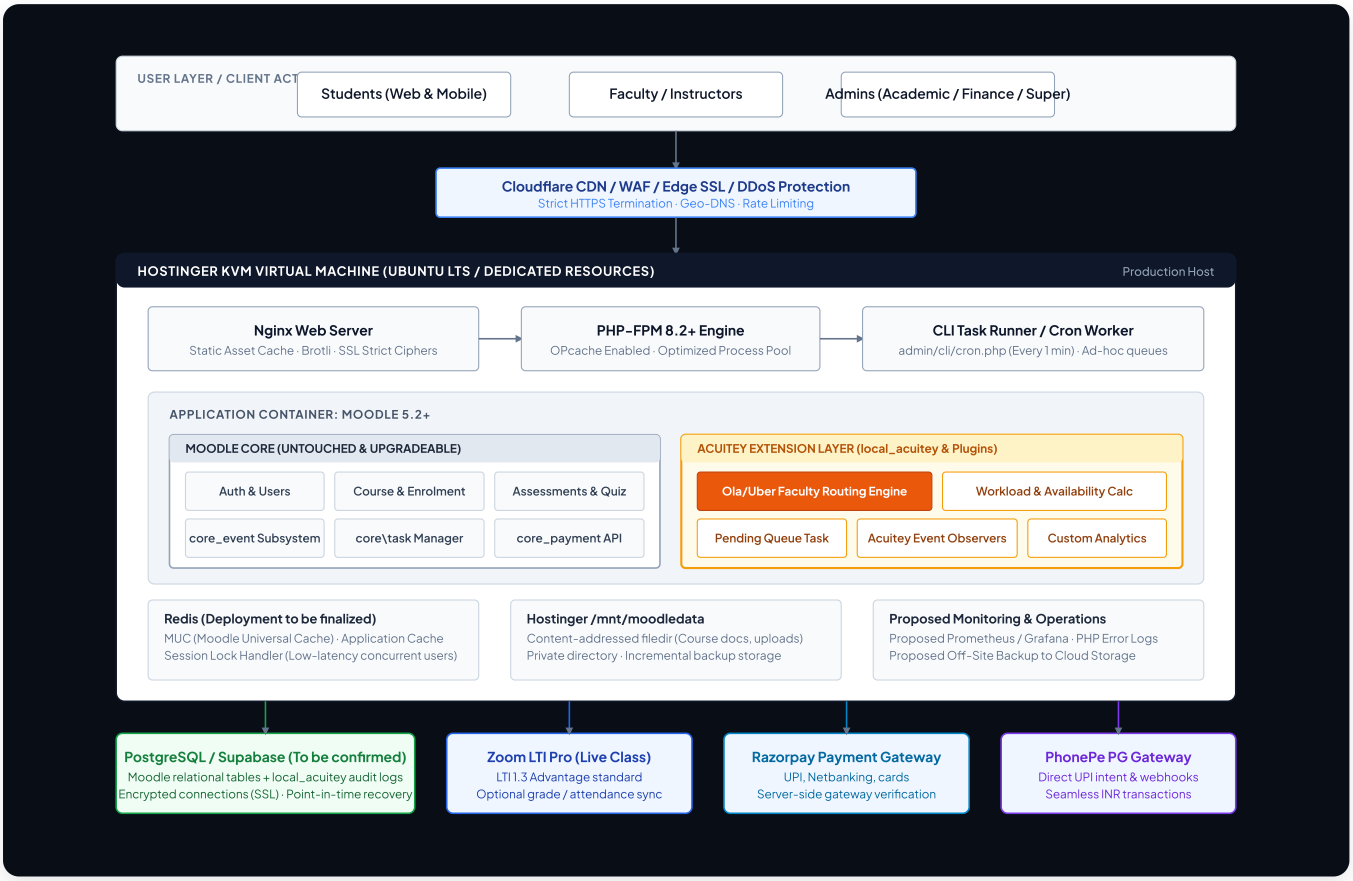
Initial System Design & Workflow Architecture

High-Concurrency Open Architecture, Automated "Ola/Uber-Style" Student-to-Faculty Dynamic Routing Engine, Native Moodle Core Extension Framework, and Enterprise Cloud Infrastructure.

DOCUMENT TYPE System Architecture Review	TARGET PLATFORM Moodle LMS 5.2.3+ (Build 20260916)
INFRASTRUCTURE Hostinger KVM • PostgreSQL 16 • Redis 7	DATE & VERSION September 2026 • Version 1.0.0

1. Executive Architecture Overview

The Acuitey Solutions Learning Management System (LMS) is architected upon **Moodle LMS 5.2.3+ (Branch 502)**, providing an open, secure, and horizontally scalable digital learning ecosystem. The platform uniquely pairs Moodle's enterprise course management and assessment engine with Acuitey's proprietary *Automated Dynamic Faculty Routing System*.



Moodle Core Untouched

Zero core modifications. The entire upstream Moodle 5.2 code remains 100% standard and directly upgradeable with zero merge friction.

Dynamic Faculty Allocation

Intelligent Ola/Uber-style algorithm dynamically evaluates specialization, real-time availability, and active workload to match students with instructors.

Proposed Cloud Topology

Proposed Cloudflare Edge perimeter, Hostinger KVM (recommended Ubuntu LTS), Nginx, PHP-FPM OPcache, Redis (configuration to be finalized), and managed PostgreSQL.

2. Architecture Principles & Governance

To guarantee continuous maintainability, data integrity, and compliance across multi-year institutional lifecycles, four architectural pillars govern all engineering decisions:

01 Moodle Core Remains 100% Untouched and Directly Upgradeable

Under no circumstances are Moodle core application files in `/lib`, `/course`, `/user`, `/enrol`, or `/admin` altered. Moodle security patches and upstream point releases are consumed directly from official git tags without bespoke regression testing of overridden core routines.

02 Strict Encapsulation Within Standard Extension Subsystems

All custom Acuitey business logic is housed within supported Moodle extension hooks: the primary orchestrator resides in `local/acuitey` (`local_acuitey`), payment gateways in `payment/gateway/`, and UX customizations within `theme/acuitey`. Custom tables utilize Moodle's XMLDB database abstraction (`mdl_local_acuitey_*`).

03 Avoidance of Unnecessary Third-Party Plugins

Third-party community plugins introduce severe supply chain risks, database bloat, and upgrade incompatibilities. Acuitey mandates native Moodle capability utilization or direct custom development in `local_acuitey`, eliminating brittle external dependencies.

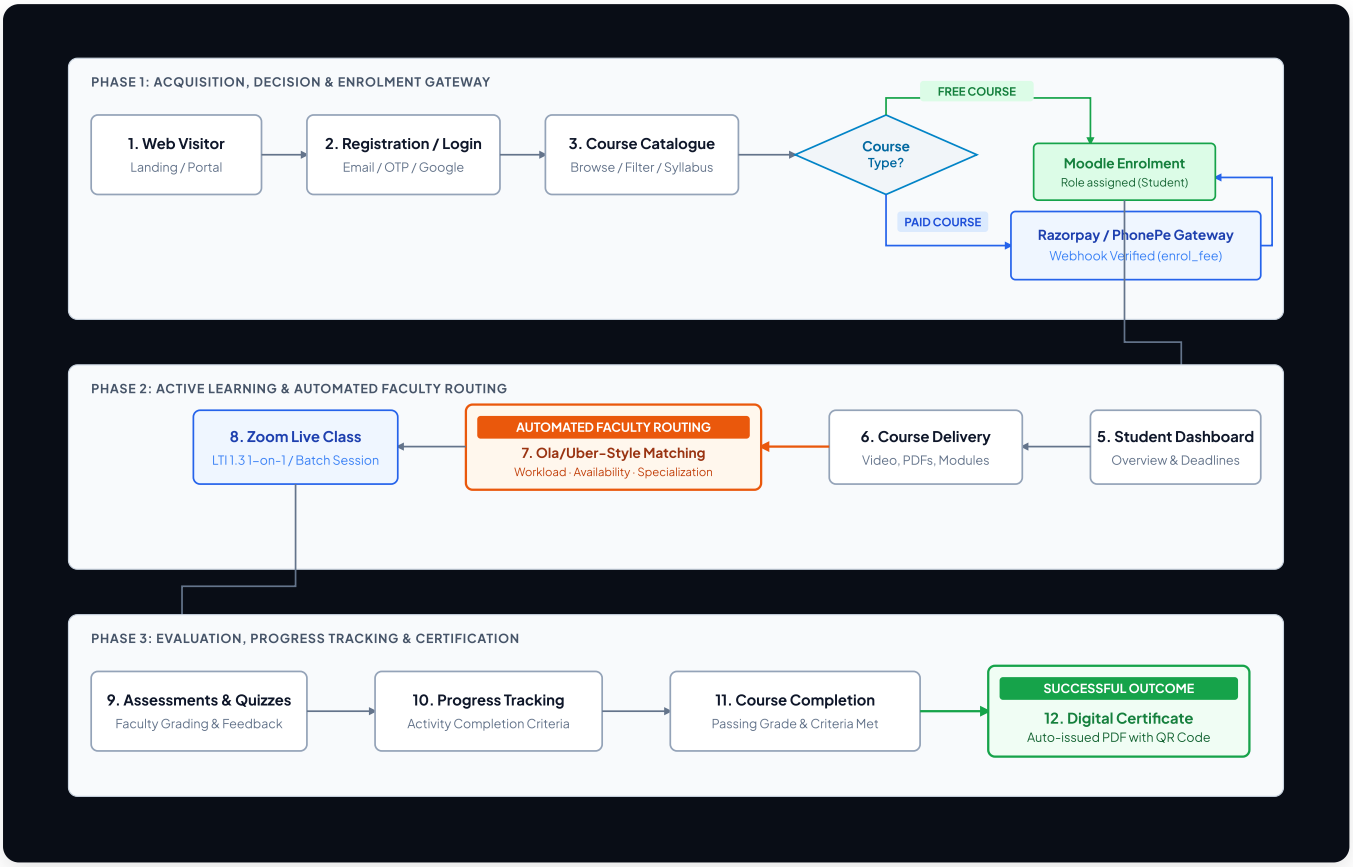
04 Asynchronous, Event-Driven & Stateless Scalability

Compute-heavy tasks—including multi-criteria faculty ranking, email dispatching, report compilation, and routing retries—are handled asynchronously through Moodle's unified Event-Observer architecture (`db/events.php`) and ad-hoc task workers (`\core\task\adhoc_task`), preventing HTTP-FPM request timeouts.

> *"Custom Acuitey functionality resides exclusively in controlled extensions. Moodle core files remain pristine, preserving long-term upgradeability and total system stability."*

3. End-to-End Student User Journey

The student user journey is designed for zero-friction exploration, rapid self-service enrolment, intelligent instructor allocation, interactive real-time learning, and automated certification.



Enrolment & Onboarding

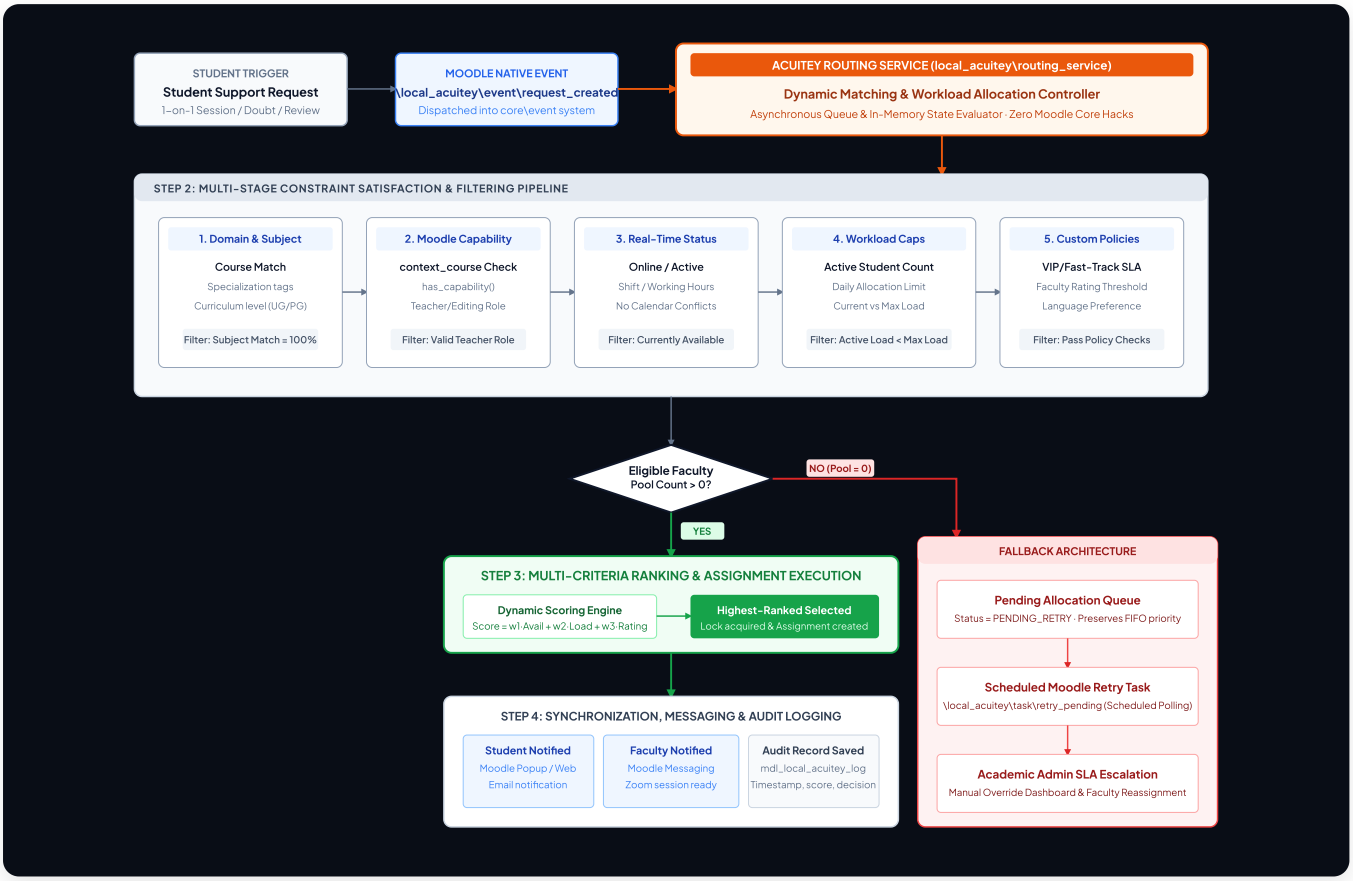
Students browse categorised curricula. Free courses grant immediate access via `enrol_self`. Paid enrolments initiate secure server-side orders via `core_payment`, instantly redirecting to dashboard upon server-side gateway-specific verification.

Dynamic Mentorship & Certification

Course entry triggers dynamic faculty routing. Students launch Zoom sessions via authenticated LTI 1.3 Advantage, submit assessments, track grade progress, and receive verifiable digital PDF completion certificates.

4. Automated Student-to-Faculty Routing Engine

The most critical custom capability in Acuitey LMS is the **Automated "Ola/Uber-Style" Student-to-Faculty Dynamic Routing Engine**. Rather than relying on static teacher course assignments, the system balances real-time pedagogical demand with instructor availability and workload capacity.



Dual Pipeline Execution: The routing engine operates an immediate synchronous evaluation pipeline for real-time requests, backed by a persistent *Pending Queue & Scheduled Retry Engine* for surge capacity management and administrative SLA escalation.

4.1 Multi-Criteria Evaluation & Fallback Workflow

The matching engine implements multi-stage filtering, dynamic mathematical scoring, and resilient exception handling to guarantee consistent service levels.

Multi-Stage Filtering Criteria

- **Subject Specialization:** Matches requested course taxonomy against faculty competency tags stored in `mdl_local_acuitey_faculty_prof`.
- **Moodle Capability Validation:** Confirms the instructor holds `moodle/course:update` and `local/acuitey:mentor` in the course context.
- **Real-Time Shift & Status:** Evaluates active shift schedules and real-time online status stored in Redis cache.
- **Workload Cap Enforcement:** Rejects faculty whose `current_workload ≥ max_workload` to eliminate instructor burnout.

Scoring & Ranking Algorithm

Eligible faculty are ranked using a multi-factor weighted equation:

```
Score = (W_spec × S_spec) + (W_load × S_load) +
        (W_perf × S_perf) + (W_tz × S_tz)
```

- **S_load (Load Balance):** Preferentially weights instructors with lowest active student counts.
- **S_perf (Performance SLA):** Incorporates historical student ratings and feedback compliance.
- **Tie-Breaking:** Round-robin rotation among equal-scoring faculty members.

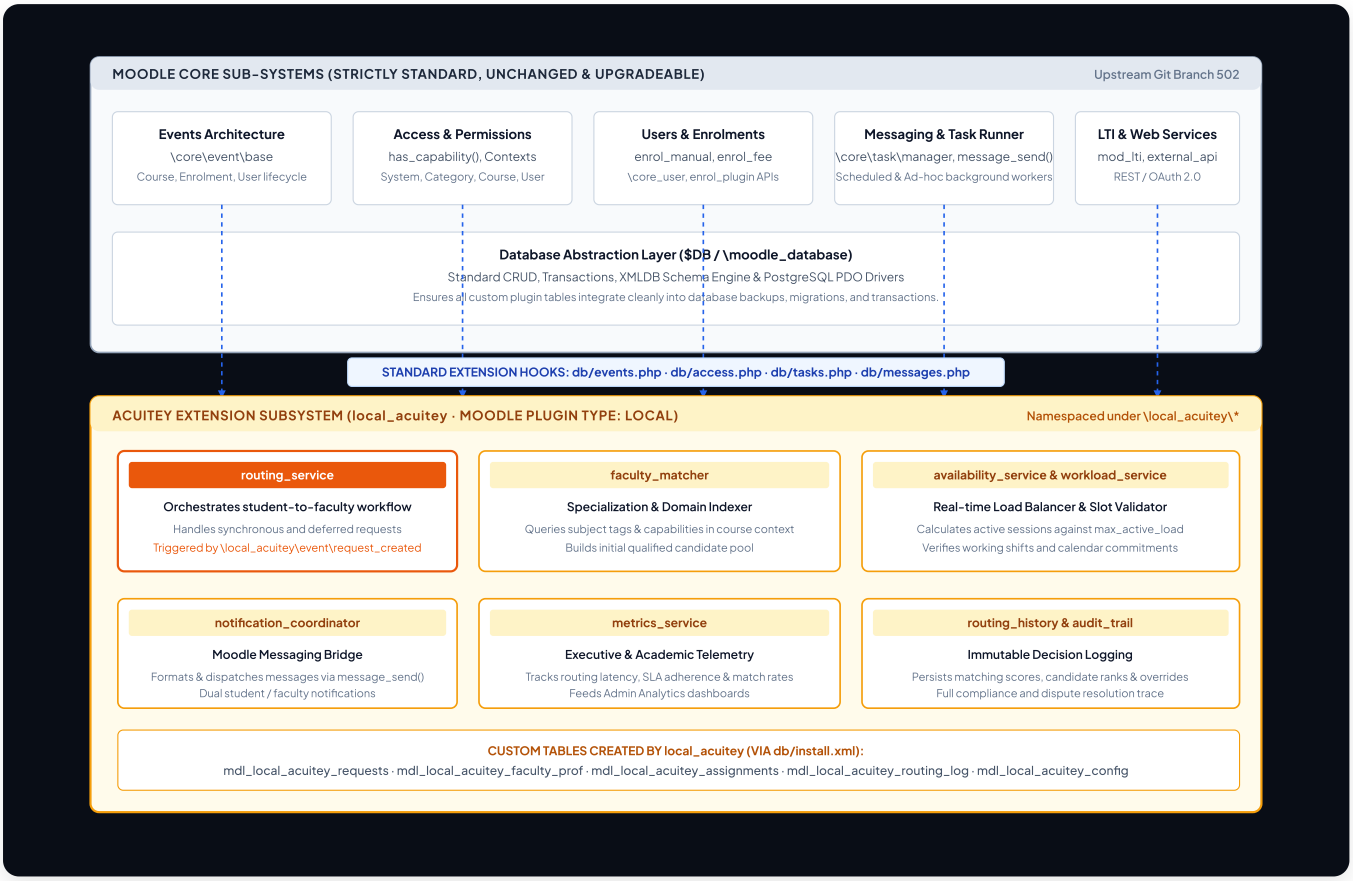
Resilient Fallback & Administrative Escalation Mechanism

If zero faculty members meet eligibility criteria during off-peak windows or sudden enrolment surges:

1. **Pending Allocation Queue:** The request is safely written to `mdl_local_acuitey_requests` with status `STATUS_PENDING`.
2. **Automated Scheduled Retry Task:** A background CLI task (`\local_acuitey\task\retry_pending_routing`) polls pending queues based on configured retry schedules.
3. **Administrative SLA Escalation:** If a request remains unassigned beyond a configured threshold, the system generates an urgent notification to the Academic Administrator console for manual reassignment.
4. **Complete Auditability:** Every evaluation cycle, calculated score, and exception is written to `mdl_local_acuitey_routing_log`.

5. Moodle Extension Strategy (local_acuitey)

Custom Acuitey functionality interfaces cleanly with native Moodle core subsystems through official extension interfaces, events, and task managers.



Events & Observers

Listens to `\core\event\user_enroled` via `db/events.php`, immediately invoking `\local_acuitey\observer\routing_observer`.

Background Tasks

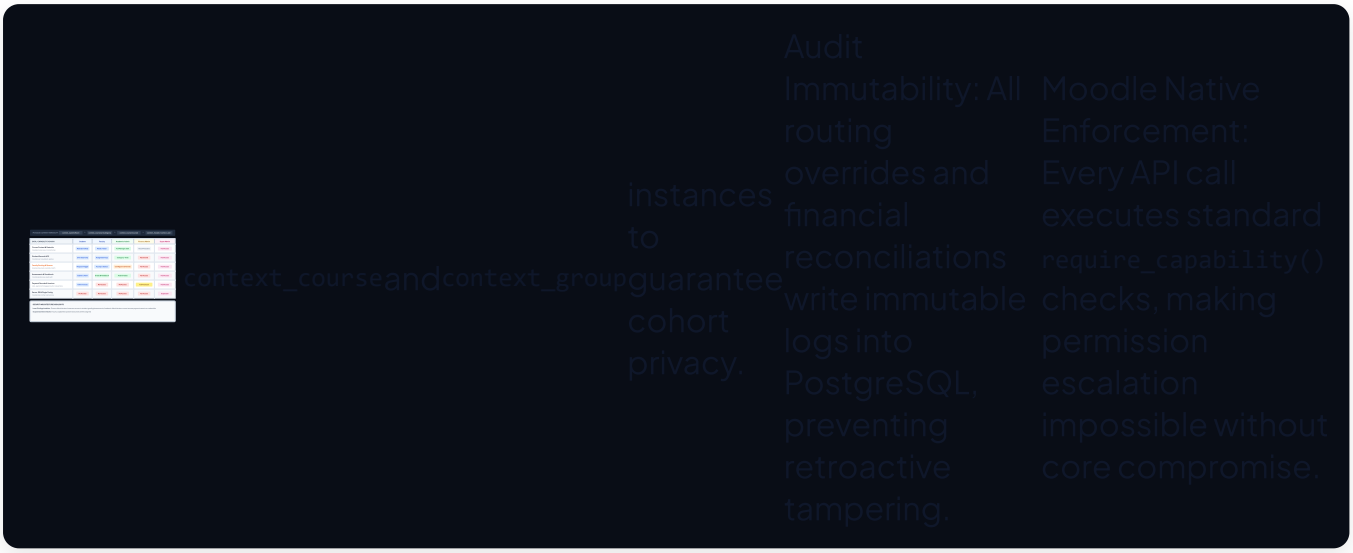
Dispatches heavy computation to `\\core\\ ask\adhoc_task`, ensuring zero latency impact on frontend student interactions.

Native Messaging

Utilizes Moodle's `message_send()` and `db/messages.php` to deliver real-time in-app, email, and mobile push alerts.

6. Contextual Roles & Security Boundaries

Security in Acuitey LMS is governed by Moodle's strict hierarchical Context Model, enforcing least-privilege role boundaries across student, faculty, and administrative tiers.



Academic & Financial Segregation

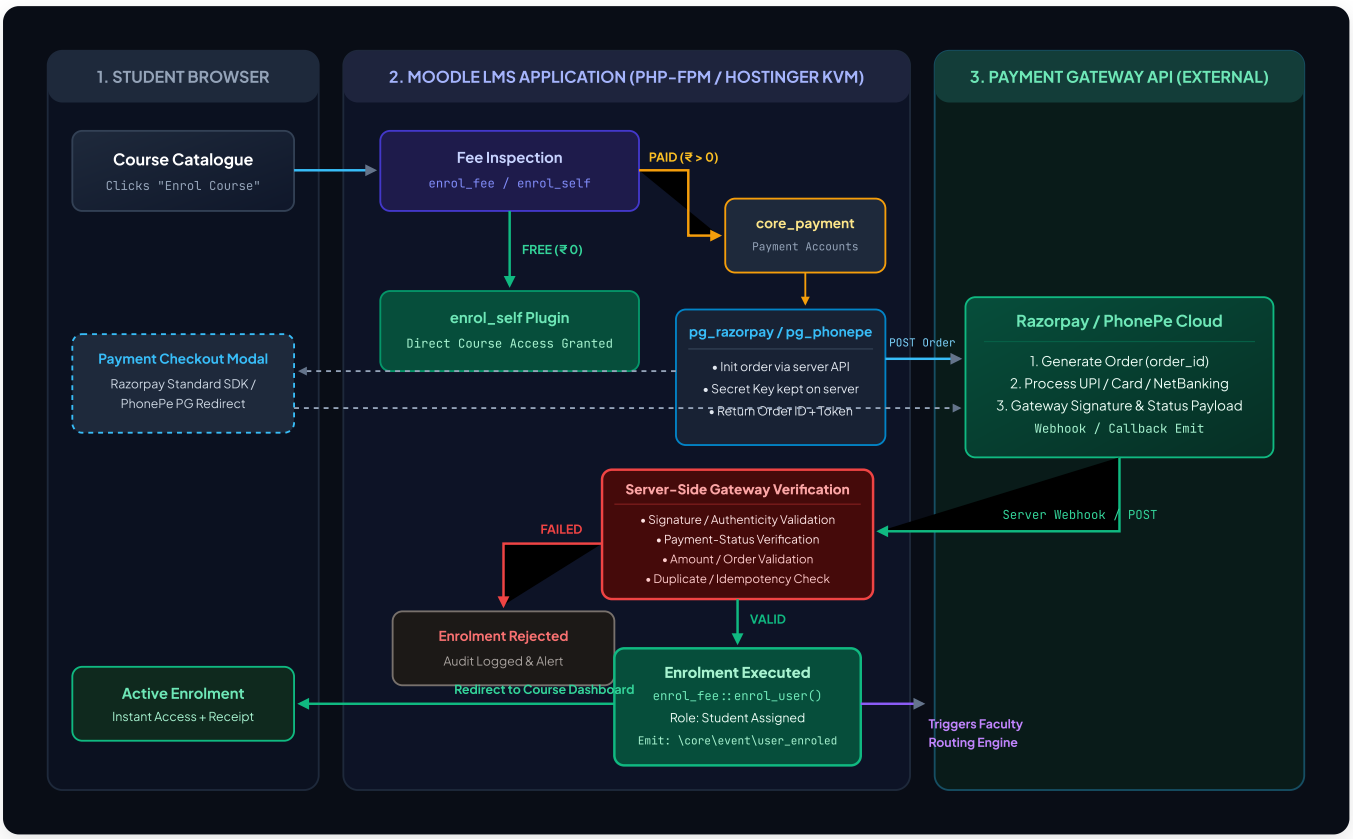
Finance Administrators have access restricted exclusively to `core_payment` records, fee reconciliations, and payout accounts. They possess zero visibility into pedagogical assessments, course content, or student grades.

Contextual Student Privacy

Students operate strictly within enrolled Course Contexts and allocated Activity Modules. Cross-student record traversal, gradebook inspection, and routing configuration modifications are blocked at the capability layer.

7. Payment Architecture & Automated Enrolment

Acuitey supports dual Indian payment gateways—**Razorpay** and **PhonePe**—interfaced through Moodle's native `core_payment` abstraction with mandatory server-side signature and payment-status validation.



Server-Side Secret Keys

Razorpay Key Secrets and PhonePe Salt/Merchant Keys reside strictly in encrypted database configurations on the Hostinger KVM. Client devices never receive private keys.

Gateway-Specific Verification

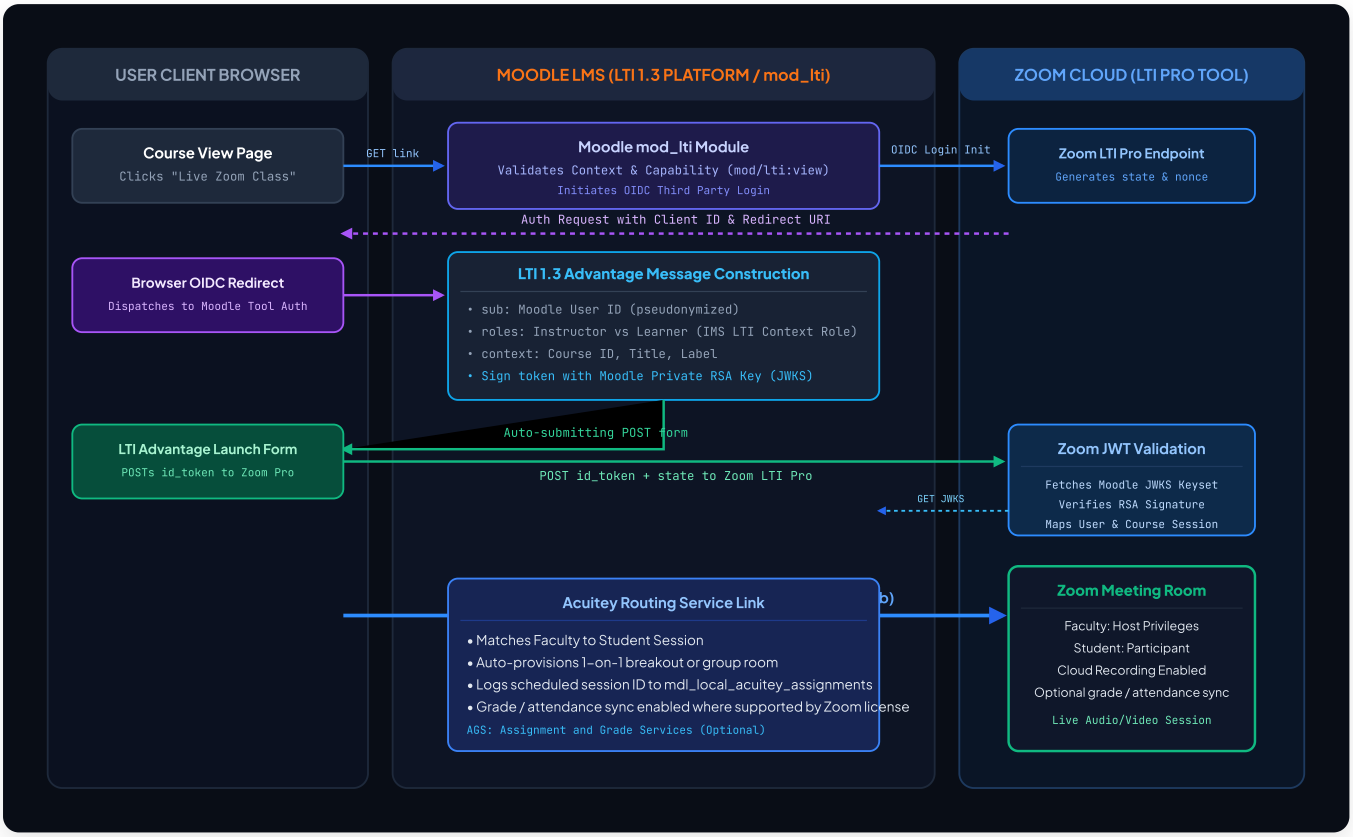
Enrolments occur exclusively when the server executes gateway-specific signature/authenticity checks, order amount verification, and payment status confirmation.

Idempotent Reconciliation

Transaction IDs are checked against `mdl_payments` prior to user enrolment, preventing duplicate course allocations or race conditions from webhook retries.

8. Zoom LTI 1.3 Advantage Integration

Interactive live learning is integrated via **LTI 1.3 Advantage (IMS Global Security Framework)** using Moodle's core `mod_lti` module and Zoom LTI Pro, eliminating passwords and unauthorized access.



Zero Password Exposure

Students and faculty never handle meeting passwords or direct Zoom account credentials. Access is cryptographically established using asymmetric RSA keypairs.

Zoombombing Prevention

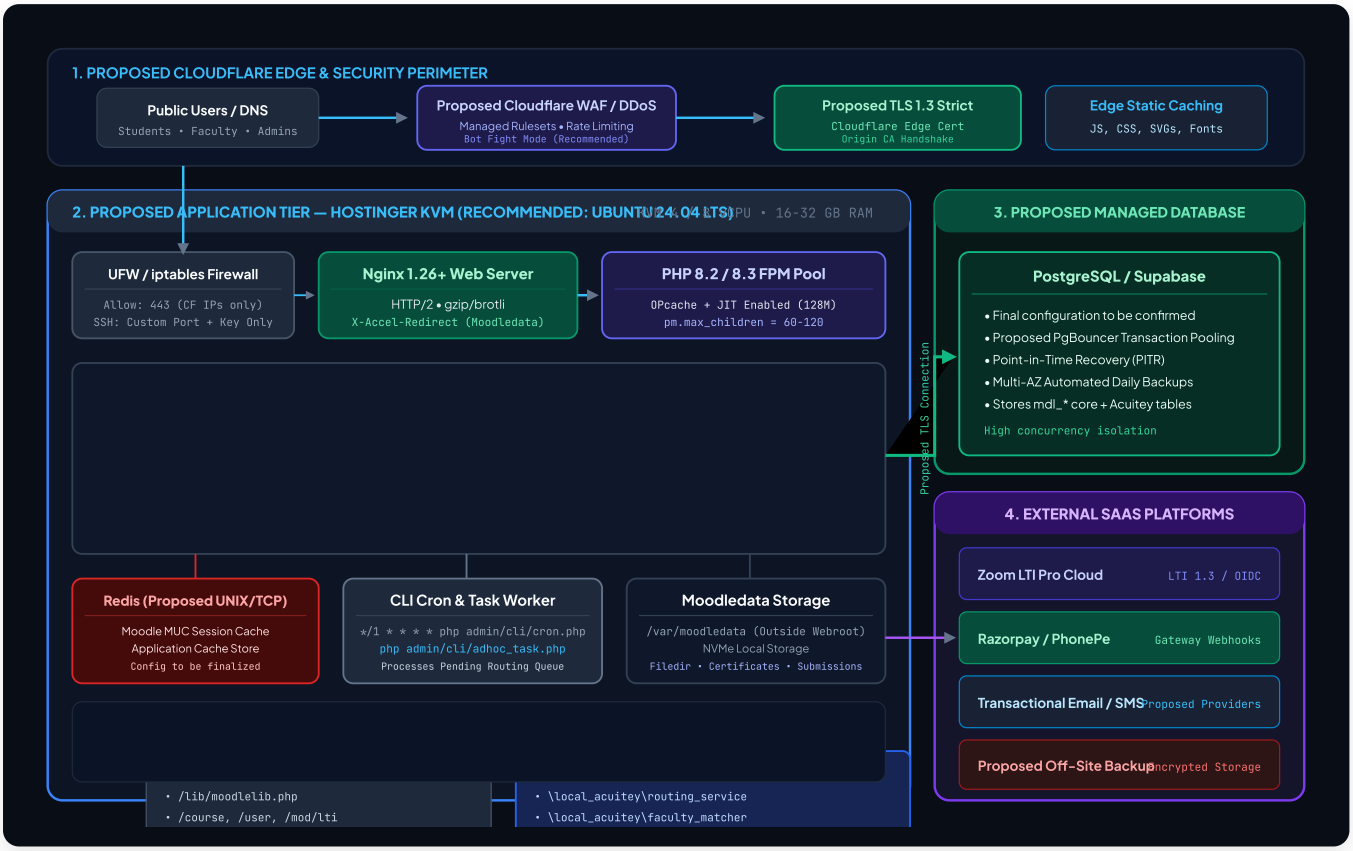
Only authenticated users with active course enrolments can generate valid LTI tokens. External or unallocated participants cannot access live sessions.

Grade & Attendance Sync

Grade and attendance synchronization may be enabled where supported by the selected Zoom LTI configuration and Acuitey's Zoom account/licensing using LTI AGS.

9. Production Infrastructure & Deployment Topology

The Acuitey LMS infrastructure is proposed on an enterprise-grade **Hostinger KVM VPS** (recommended: Ubuntu 24.04 LTS), protected by **Proposed Cloudflare Edge**, and backed by a proposed managed **PostgreSQL / Supabase** database tier (final configuration to be confirmed).



Proposed Cloudflare Edge

Recommended perimeter for terminating TLS 1.3, mitigating DDoS attacks, enforcing WAF rules, and caching static assets.

Proposed Hostinger KVM

Dedicated compute running Nginx, PHP-FPM 8.2/8.3, Redis (configuration to be finalized), and CLI cron task workers.

PostgreSQL / Supabase

Managed PostgreSQL database tier (final configuration to be confirmed), connection pooling, and automated backups.

Proposed Operations

Proposed monitoring layer (Prometheus/Grafana), automated snapshots, off-site backup storage, and SSH key controls.

10. Native Moodle vs. Acuitey Custom Development

To provide executive clarity, the following matrix delineates standard Moodle platform capabilities from custom Acuitey proprietary engineering:

Functional Area	Native Moodle 5.2.3+ Infrastructure	Acuitey Custom Engineering	Architecture Designation
User Authentication	Standard `auth_*` core plugins & manual accounts	Custom profile fields & taxonomy tags	NATIVE MOODLE
Role & Security RBAC	Contextual RBAC (System → Category → Course)	Custom capability definitions in `db/access.php`	NATIVE MOODLE
Course Catalogue	Course categories, descriptions, and tags	Custom responsive UI grid in `theme_acuitey`	HYBRID
Enrolment Processing	`enrol_manual`, `enrol_self`, `enrol_fee`	Automated event trigger invocation	NATIVE MOODLE
Payment Gateway API	Moodle `core_payment` abstraction layer	Razorpay & PhonePe gateway plugins	ACUITEY CUSTOM
Payment Verification	Transaction logging framework	Server-side gateway-specific signature and status check	ACUITEY CUSTOM
Student Dashboard	Moodle `/my/` dashboard & core course blocks	Personalized allocation & mentoring widgets	HYBRID
Faculty Routing Engine	*Not provided in Moodle core*	Ola/Uber multi-criteria dynamic matching service	ACUITEY PROPRIETARY
Faculty Availability Tracking	*Not provided in Moodle core*	Real-time shift schedules & Redis presence state	ACUITEY PROPRIETARY
Workload Balancing	*Not provided in Moodle core*	Capacity counters & dynamic ranking weights	ACUITEY PROPRIETARY
Pending Allocation Queue	*Not provided in Moodle core*	Ad-hoc background retry task & admin escalation	ACUITEY PROPRIETARY
Assessments & Quizzes	Moodle `mod_assign`, `mod_quiz`, rubrics	Standard configuration & grade templates	NATIVE MOODLE
Live Video Conferencing	Core `mod_lti` LTI 1.3 Advantage Platform	Zoom LTI Pro configuration (Optional AGS sync)	INTEGRATION
Event Observers	Moodle `\core\event\base` subsystem	Custom routing events & listeners	HYBRID
Notification Dispatch	Core `message_send()` & provider framework	Custom message templates in `db/messages.php`	HYBRID
PDF Certificates	Core certificate & completion tracking	Branded PDF layout & cryptographic verification	NATIVE MOODLE

11. Implementation Sequence & Execution Roadmap

The platform rollout follows a phased, 6-stage engineering sequence engineered to de-risk core dependencies, establish security baselines, and rigorously validate the dynamic routing engine prior to go-live.

STAGE 01 **Infrastructure Provisioning & Hardening**
Deploy Hostinger KVM (Recommended Ubuntu LTS), Nginx, PHP-FPM with OPcache, local Redis. Provision PostgreSQL / Supabase with connection pooling. Configure proposed Cloudflare edge security rules.

STAGE 02 **Moodle Core Foundation & Payment Gateway Setup**
Install untouched Moodle 5.2.3+ codebase. Configure MUC Redis caching and CLI cron task runner. Implement custom Razorpay and PhonePe gateway plugins with server-side gateway-specific verification.

STAGE 03 **Acuitey Routing Engine Development (local_acuitey)**
Create XMLDB table schemas (`mdl_local_acuitey_*`). Implement faculty matching algorithm, availability checking, and workload balancing services. Build background retry processor and pending queue.

STAGE 04 **Zoom LTI 1.3 Advantage & Academic Workflows**
Establish cryptographic OIDC/JWT handshake between Moodle `mod_lti` and Zoom LTI Pro. Configure course category taxonomies, assessment rubrics, gradebook aggregations, and Acuitey custom theme.

STAGE 05 **Reporting, Administrative Consoles & Observability**
Build Academic Administrator routing control dashboards and Finance payment reconciliation tools. Deploy proposed Prometheus/Grafana monitoring dashboards for real-time infrastructure telemetry.

STAGE 06 **End-to-End Testing, Pilot Cohort & Production Go-Live**
Conduct automated high-concurrency stress testing of routing algorithms. Execute end-to-end security audits. Train faculty and administrative staff. Execute production cutover and launch.

12. Architecture Confirmation & Design Principles

This architecture document confirms compliance with enterprise software engineering standards, Moodle architectural guidelines, and approved institutional design principles.

Approved Architectural Principles & Declarations

Moodle Core Integrity Statement:

> *"The Acuitey automated student-to-faculty workflow will be implemented using Moodle's supported extension architecture. Moodle core files will not be modified for this functionality. Moodle APIs, events, capabilities, tasks and messaging mechanisms will be used where appropriate, while Acuitey-specific routing logic will remain isolated within controlled custom extensions."*

Third-Party Plugin Governance Statement:

> *"Third-party Moodle plugins will only be introduced where they provide a justified and maintainable capability that is not appropriately delivered through Moodle native functionality or the Acuitey custom extension."*

Architecture Confirmation Details

Architecture Lead: Enterprise Solutions Architect	**Status:** Approved for Implementation Baseline
Target Moodle Release: 5.2.3+ (Build 20260916)	**Proposed Operating System:** Ubuntu LTS (Hostinger KVM)
Primary Custom Plugin: `local/acuitey`	**Review Date:** September 2026

Document Status: This specification represents the agreed design principles for all Acuitey LMS engineering activities. Any subsequent modifications to core subsystems or integration boundaries require architectural review.